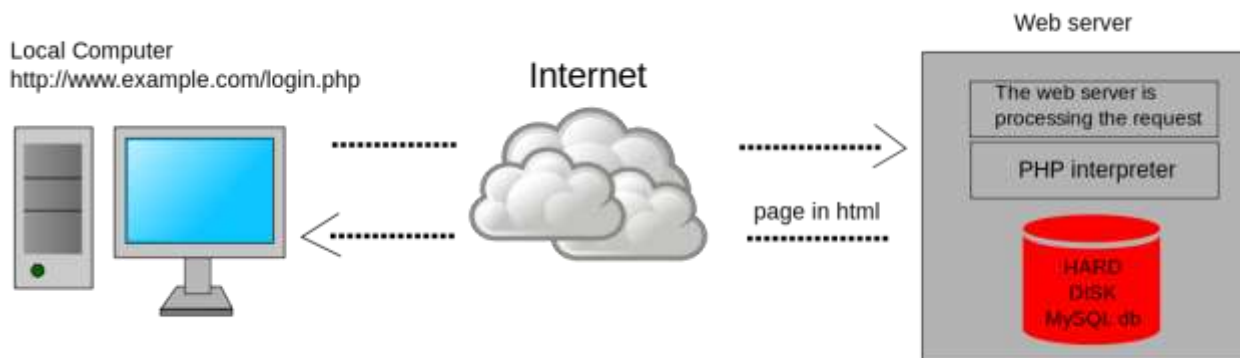


Proiectarea aplicațiilor Web (PAW)

Introducere

PHP este un limbaj de programare realizat în principal pentru realizarea site-urilor web dinamice. PHP este un acronim recursiv pentru PHP: Hypertext Preprocessor. A fost dezvoltat în 1994 și este unul din cele mai utilizate limbaje de scripting pentru programare web (conform Tiobe Index PHP ocupă locul 6 ca și popularitate după Java, C, C++, C# și Python). Din 2014 limbajul PHP are o specificație formală - PHP Language Specification (<https://github.com/php/php-langspect>).



Sintaxa limbajului PHP provine din sintaxa limbajelor C, Perl și Java.

Un exemplu de script PHP:

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Exemplu PHP</title>
  </head>
  <body>

    <?php
    echo "Primul exemplu de script PHP!";
    ?>

  </body>
</html>
```

Codul sursă PHP este încorporat între instrucțiuni de procesare de început și de sfârșit speciale <?php și ?>.

Codul PHP este executat pe server, generând HTML care este apoi trimis către client. Clientul va primi rezultatele rulării acelui script, fără a putea cunoaște codul-sursă ce stă la bază.

Rezultatul execuției script-ului de mai sus

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Exemplu PHP</title>
  </head>
  <body>
    <p>Primul exemplu de script PHP!</p>
  </body>
</html>
```

Construcția de limbaj echo

Construcția de limbaj **echo** afișează toți parametrii primiți ca parametri (echo nu este o funcție).

```
void echo ( string $arg1 [, string $... ] )
```

Nefiind o funcție nu este obligatorie utilizarea parantezelor.

Construcția **echo** are și o formă scurtă `<?=$var ?>`. Această formă funcționează doar directiva `short_open_tag` este true în fișierul de configurare `php.ini`.

Construcția de limbaj print

```
int print ( string $arg )
```

Construcția de limbaj **print** afișează un șir de caractere primit ca parametru. Valoarea întoarsă este întotdeauna 1.

```
<?php  
print("Un script PHP");  
print "print() functioneaza fara parametri";  
?>
```

Domeniile principale de utilizare ale limbajului PHP:

- server side scripting (scripting la nivel de server) - domeniul de bază al utilizării limbajului PHP - serverul web cu o instalare PHP conectată la el (interpretorul PHP în calitate de CGI, sau modul pentru server).
- shell scripting (scripting în linia de comandă) - interpretorul PHP în linie de comandă
- aplicațiilor desktop - extensia PHP-GTK pentru dezvoltarea de aplicații ce vor rula pe mai multe platforme.

PHP poate fi utilizat pe toate sistemele de operare majore, inclusiv Linux, Unix (inclusiv HP-UX, Solaris și OpenBSD), Microsoft Windows, Mac OS X, RISC OS și altele. PHP de asemenea susține majoritatea serverelor web existente în prezent. Acestea includ Apache, IIS, și multe altele. De asemenea se includ și serverele ce pot utiliza binarul PHP FastCGI, cum ar fi `lighttpd` și `nginx`. PHP funcționează ca modul sau ca procesor CGI.

PHP se poate utiliza cu programarea procedurală sau cu programarea orientată pe obiecte (POO), sau cu o combinație a acestora.

PHP se poate folosi nu doar la afișarea HTML ci și la afișarea de imagini, fișiere PDF. PHP poate genera automat aceste fișiere și le poate salva în sistemul de fișiere.

Una dintre cele mai puternice și semnificative facilități ale PHP este susținerea unui larg domeniu de baze de date. Scrierea unei pagini web ce accesează o bază de date este incredibil de simplă utilizând una din extensiile de lucru cu baze de date (de ex. pentru `mysql`), sau utilizând un nivel de abstractizare precum `PDO`, sau conectarea la orice bază de date ce susține standardul "Open Database Connection" cu ajutorul extensiei `ODBC`.

PHP dispune de funcții și clase pentru utilizarea de protocoale cum ar fi LDAP, IMAP, SNMP, NNTP, POP3, HTTP, COM (pe Windows) și multe altele. Se pot realiza aplicații de rețea prin utilizarea de socket-uri și prin utilizarea de orice alt protocol. PHP susține WDDX - schimbul de date complexe între aproape toate limbajele de programare web. Vorbind despre interconectare, PHP susține instanțierea obiectelor Java și utilizarea lor transparentă ca obiecte PHP.

PHP posedă facilități de procesare a textului foarte utile, care includ expresii regulate compatibile cu Perl (PCRE) și multe extensii și instrumente pentru accesarea și interpretarea documentelor XML. PHP standardizează toate extensiile XML pe baza solidă a bibliotecii libxml2 și extinde setul de facilități adăugând susținerea SimpleXML, XMLReader și XMLWriter.

Sintaxa de bază

Tag-uri PHP

```
<?php
/* forma recomandata */
?>
```

```
<?
/* taguri scurte, acceptarea lor de catre web-server depinde de configurare */
?>
```

```
<?=$var
/* taguri scurte, acceptarea lor de catre web-server depinde de configurare */
?>
```

Tag-urile ASP <%, %>, <%= și tag-ul de stil script <script language="php"> sunt excluse din PHP versiunea 7.

```
<script language="php">
/* forma disponibila oricand, putin folosita */
</script>
```

```
<%
/* taguri in stilul asp, acceptarea lor de catre web-server depinde de configurare
*/
%>
```

```
<%= $var;
%>
```

Dacă fișierul conține doar cod PHP se poate omite tagul de sfârșit.

```
<?php
echo "Prima declaratie";
// ... cod PHP
echo "Ultima declaratie";
// script-ul se termina aici fara tag de inchidere
```

Separarea instrucțiunilor

PHP cere ca instrucțiunile să fie terminate cu punct și virgulă precum în C sau Perl.

```
<?php
    echo 'Acesta este un test';
?>
```

```
<?php echo 'Acesta este un test' ?>
```

```
<?php echo 'Tag-ul de inchidere omis';
```

Comentarii

PHP suportă comentarii 'C', 'C++' sau stil Unix shell (stil Perl). De exemplu:

```
/*
comentariu multi rand
stil C
*/
```

```
// comentariu de un singur rand stil C++
```

```
# comentariu de un singur rand stil Unix shell
```

Exemplu:

```
<?php
    echo 'Acesta este un test'; // Acesta este un comentariu de un rand tip C++
    /* Acesta este un comentariu multi-rand
       un nou rand de comentariu */
    echo 'Acesta este un alt test';
    echo 'Un test final'; # Acesta este un comentariu de un rand stil shell
?>
```

Tipuri de date

PHP suportă 8 tipuri primitive de date.

Tipul boolean

O expresie booleană exprimă o valoare de adevăr și poate fi TRUE sau FALSE. Aceste constante sunt case-insensitive.

Exemplu:

```
<?php
$var = TRUE; // atribuie valoarea TRUE variabilei $var
var_dump($var);
?>
```

Conversia la tipul boolean

Pentru a converti o valoare explicit la tipul boolean se folosește operatorul cast (bool) sau (boolean).

O valoare este convertită automat dacă un operator, o funcție sau structură de control necesită un argument boolean.

Pentru conversia la tipul boolean următoarele valori sunt considerate valoarea boolean FALSE:

- valoarea boolean FALSE
- valoarea întreagă 0 (zero)
- valoarea reală (float) 0.0
- șirul de caractere vid "" și șirul "0"
- un vector cu zero elemente
- tipul de date NULL (și variabilele care nu au valori setate)
- obiecte SimpleXML create din taguri vide

Toate celelalte valori sunt considerate TRUE (inclusiv resource și NAN).

Tipul Integer

Un întreg este un număr din mulțimea $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$.

Întregii pot fi specificați:

- în baza 10
- în baza 16 (0x)
- în baza 8 (0)
- în baza 2 (0b)

Formal:

```
decimal      : [1-9] [0-9] *
              | 0
```

hexadecimal : 0[xX][0-9a-fA-F]+

octal : 0[0-7]+

binary : 0b[01]+

integer : [+]?decimal
| [+]?hexazecimal
| [+]?octal
| [+]?binar

```
<?php
$a = 1234; // numar intreg
$a = -123; // numar negativ
$a = 0123; // numar in baza 8 (83 in baza 10)
$a = 0x1A; // numar hexazecimal (26 in baza 10)
$a = 0b11111111; // numar binar (255 in baza 10)
?>
```

```
<?php
$var = 1234;
var_dump($var);
?>
```

Funcția **var_dump** întoarce tipul de date al variabilei și valoarea acesteia.

Conversia la tipul întreg

Pentru a converti o valoare explicit la tipul întreg se folosește operatorul cast (int) sau (integer).

Conversia de la tipul boolean

FALSE se convertește la 0, și TRUE se convertește la 1.

Conversia de la tipul real

Când se face conversia de la tipul real la număr întreg, numărul va fi rotunjit spre zero.

Numere reale

Numerele reale (cunoscute ca "float", "double", sau "real number") pot fi specificate folosind sintaxa:

```
<?php
$a = 1.234;
$b = 1.2e3;
$c = 7E-10;
var_dump($a);
var_dump($b);
var_dump($c);
?>
```

Formal:

```
LNUM      [0-9]+
DNUM      ([0-9]*[\.]{LNUM}) | ({LNUM}[\.][0-9]*)
EXPONENT_DNUM  [+]?((({LNUM} | {DNUM}) [eE] [+]? {LNUM})
```

Dimensiunea tipului real este dependentă de platformă, valoarea maximă este ~1.8e308 cu o precizie de 14 cifre zecimale (formatul IEEE pe 64 biți).

Tipul string

O variabilă de tip string este un șir de caractere (o secvență de caractere). Un caracter se reprezintă pe un octet, deci sunt 256 de caractere distincte. Acest lucru implică faptul că interpretorul PHP nu oferă suport nativ pentru setul de caractere Unicode. Lungimea variabilelor de tip string nu este limitată de către interpretor.

Literalii de tip șir de caractere pot fi specificați în patru moduri diferite.

Șiruri de caractere delimitate de apostrofuli (ghilimele simple) - caracterul '

Pentru a specifica un singur apostrof ('), aceasta trebuie înlocuit cu un backslash (\). Pentru a specifica un backslash, se folosește dubla (\\). Toate celelalte cazuri de backslash vor fi tratate ca un backslash literal: aceasta înseamnă că celelalte secvențe escape cum ar fi \n \r sau, vor fi redată literalmente așa cum se specifică și nu cu o semnificație specială.

Spre deosebire de sintaxele dublu-citat și heredoc, variabilele și secvențele escape de caractere speciale nu vor fi extinse atunci când acestea apar în șiruri delimitate cu apostrof.

Exemplu:

```
<?php
echo 'un simplu string';

echo 'Sirul poate
fi pe mai multe
randuri';

// Rezultat: "I'll be back by Monday"
echo '"I\'ll be back by Monday"';

// Rezultat: Do you want to delete files C:\Windows\*. *?
echo 'You deleted C:\\Windows\\*. *?';

// Rezultat: Nu va expanda secventa \n in rand nou
echo 'Nu va expanda secventa \n in rand nou';

// Rezultat: Variabila $var nu este expandata
echo 'Variabila $var nu este expandata';
?>
```

Șiruri de caractere delimitate de ghilimele duble

Șirurile delimitate prin ghilimele duble au particularitatea ca pot interpreta variabilele și caracterele speciale din interiorul lor. Astfel, la evaluarea textului, variabilele existente în interiorul lui sunt înlocuite cu valoarea lor iar rezultatul final este returnat.

Exemplu:

```
<?php
$v = "o valoare"; // variabila v
echo "Textul definit cu ghilimele contine $a";
// se va afisa
// Textul definit cu ghilimele contine o valoare
?>
```

Folosind această notăție, pot fi specificate mai multe caractere speciale, pe lângă caracterele de la varianta anterioară, printre care:

- a) sfârșit de linie ("r")
- b) rând nou ("n")
- c) tab orizontal ("t")
- d) semnul dolar ("\$")
- e) ghilimelele duble ("")
- f) secvențe de caractere pentru specificarea expresiilor regulate în notăție octală ("[0-7]{1,3}")
- g) secvențe de caractere pentru expresiile regulate în notăție hexazecimală ("x[0-9A-Fa-f]{1,2}").
- h) secvențe de caractere pentru expresiile regulate în notăție UTF-8 ("u{[0-9A-Fa-f]+}").

Notăția heredoc (notăția specială <<<)

Pentru a specifica un șir de caractere folosind această notăție, trebuie utilizat operatorul "<<<" urmat de un identificator ales de utilizator. Toate caracterele care se află între operatorul "<<<", urmat de un identificator pe o singură linie, și același identificator pe o altă linie, vor constitui valoarea șirului de caractere.

Aceste șiruri de caractere au avantajul că nu necesită marcarea (escaparea) delimitatorilor prin \' sau \". În rest, aceste stringuri sunt tratate în același mod ca cele delimitate de ghilimele duble, în sensul că permit interpretarea variabilelor și a altor caractere speciale.

Exemplu:

```
<?php
$v = 'test; // definesc o variabila

echo <<<EOT
Un text pe mai multe linii.
Apoi se afiseaza $v
EOT;
?>
```

Opțional poate fi și în forma

```
<?php
$v = 'test; // definesc o variabila

echo <<<"EOT"
Un text pe mai multe linii.
Apoi se afiseaza $v
EOT;
?>
```

Notăția nowdoc (notația specială <<<)

În versiunile mai noi de PHP (de la 5.3.0) a fost introdusă posibilitatea de a defini șiruri prin notația specială <<< fără a interpreta variabilele conținute.

Exemplu:

```
<?php
$v = 'o valoare'; // definesc o variabila

echo <<<'EOT'
Variabilele si caracterele speciale nu sunt interpretate.
$v si \n raman asa cum sunt.
EOT;
?>
```

Pentru a accesa un anumit caracter din șirul de caractere, se folosește, după numele variabilei de tip string, indicele caracterului care trebuie accesat scris între acolade. De exemplu, \$str{0} returnează primul caracter din șirul de caractere \$str.

În cazul în care dorim să concatenăm două șiruri de caractere, vom folosi operatorul ".".

Există posibilitatea de a converti o variabilă de orice tip la tipul string. Pentru șirurile de caractere, se pot efectua următoarele conversii:

- valoarea logică TRUE va fi convertită la șirul "1", iar valoarea logică FALSE va fi convertită la șirul vid ("");
- un număr întreg va fi convertit la un șir de caractere care reprezintă valoarea numărului în baza 10;
- un număr real va fi convertit la un șir de caractere care reprezintă notația științifică a acestuia;
- obiectele sunt întotdeauna convertite la șirul "Object";
- variabilele de tipul resource sunt convertite la șirul "Resource id #n", unde n reprezintă un număr unic atașat resursei respective de către interpretorul PHP;
- valoarea NULL este convertită la șirul vid ("").

Tipul resource

Variabilele de tip resource sunt folosite pentru păstrarea unor referințe către anumite resurse externe cum ar fi conexiuni la baze de date, fișiere, etc.

Resursele sunt create și utilizate de anumite funcții speciale. Datorită specificului acestui tip de date, valoarea nici unei variabile de alt tip nu poate fi convertită la tipul resource.

Tipul NULL

Valoarea specială NULL este atribuită oricărei variabile care nu a fost încă inițializată. Această valoare este singura pe care o pot avea variabilele de tip NULL.

Se consideră că o variabilă are tipul NULL dacă:

- i s-a atribuit constanta NULL;
- nu a fost inițializată
- a fost dezințializată (prin intermediul funcției unset ()).

Exemplu:

```
<?php
$var = NULL;
echo $var;
var_dump($var);
?>
```

```
<?php
$nume = 'Ion';
var_dump(unset($nume));
var_dump($nume);
?>
```

Variabile

Variabilele în PHP se reprezintă prin intermediul unui identificator prefixat de caracterul "\$".

Numele variabilei este case-sensitive (de exemplu variabila \$var este diferită de \$VAR) și este compus din caractere alfa-numerice și simbolul "_".

Tipul unei variabile se determină funcție de valoarea pe care o stochează. Variabilele se pot folosi fără a trebui să fie declarate în prealabil.

Numele variabilei poate fi orice identificator valid (un text ce conține doar litere, cifre, și underscore, fără spații sau alte caractere; un identificator nu poate începe cu o cifră).

În PHP declararea variabilelor se realizează în același timp cu inițializarea lor (cu atribuirea unei valori de început). Atribuirea unei valori se realizează cu ajutorul instrucțiunii de atribuire ce are forma <nume variabila> = <valoare>;

Constante

Constantele pot fi considerate variabile ale căror valori sunt fixate și nu mai pot fi alterate pe parcursul programului. O constantă stochează o valoare, precum face și o variabilă, dar această valoare nu mai poate fi modificată în script, după ce a fost stabilită (i.e. constanta a fost definită). Pentru a defini o constantă se va folosi construcția sau funcția define(). Numele constantelor va fi scris, de obicei cu majuscule.

Această opțiune nu este obligatorie, însă este recomandabilă deoarece va face codul mai frumos și mai lizibil. O diferență importantă între constante și variabile, este faptul că o constantă nu are în fața ei semnul \$.

Exemplu:

```
<?php
define('NUME', 'Popescu Ion');
define('VARSTA', '20 de ani');
echo "Numele persoanei este".NUME.' si varsta '.VARSTA;
?>
```

Definirea constantelor se poate face de la versiunea PHP 5.3 și folosind cuvântul cheie **const**:

Exemplu:

```
<?php
// de la PHP 5.3.0
const CONSTANT = 'o valoare constanta';

echo CONSTANT;
?>
```

Exemple de constante predefinite: PHP_VERSION, PHP_OS.

Expresii în PHP

Expresiile sunt combinații valide de operanzi și operatori. Operanzii pot fi variabile, constante literali sau alte expresii.

Operatori

Operatorii aritmetici (+, -, *, /, %) sunt cei cunoscuți din limbajele C/C++, Java.

Exemple	Nume	Rezultat
\$a + \$b	Adunare	Suma lui \$a cu \$b.
\$a - \$b	Scădere	Diferența dintre \$a și \$b.
\$a * \$b	Înmulțire	Produsul lui \$a cu \$b.
\$a / \$b	Împărțire	Câtul împărțirii lui \$a la \$b.
\$a % \$b	Modulo	Restul împărțirii lui \$a la \$b.

Operatorii de asignare ("=" sau "+=")

Exemplu de utilizare operatorilor de asignare "=", "+=" și concatenare ".="

```
<?php
//Operatori de asignare "=" sau "+="
$a = ($b = 4) + 5; // $a este egal cu 9 si $b a fost setat la 4.
echo "a=", $a, " ", "b=", $b;
$a = 3;
$a += 5; // seteaza $a la 8, ca si in expresia: $a = $a + 5;
$b = "Salut ";
```

```
$b .= "PHP!"; // seteaza $b la "Salut PHP!", ca si in $b = $b . "PHP!";
echo "a=", $a, "<br />", "b=", $b;
?>
```

Operatorii la nivel de bit (&, |, ^, ~, <<, >>)

Exemple	Nume	Rezultat
\$a & \$b	And	Biții setați la 1 în \$a și în \$b vor fi setați la 1. („și” bit cu bit)
\$a \$b	Or	Biții setați la 1 fie \$a, fie în \$b vor fi setați la 1. („sau” bit cu bit)
\$a ^ \$b	Xor	Biții setați 1 fie \$a, fie în \$b dar nu simultan vor fi setați 1. („sau exclusiv”)
~ \$a	Not	Biții setați la 1 în \$a, vor fi resetați la 0, și vice versa. („not” bit cu bit)
\$a << \$b	Shift left	Deplasează \$b poziții spre stânga biții lui \$a.
\$a >> \$b	Shift right	Deplasează \$b poziții spre dreapta biții lui \$a.

Operatorii relaționali (ca noutate se poate menționa aici „==” care are ca rezultat constanta TRUE dacă operanzii au valori identice și în plus sunt de același tip)

Exemple	Nume	Rezultat
\$a == \$b	Egalitate	TRUE dacă \$a este egal cu \$b.
\$a === \$b	Identitate	TRUE dacă \$a este egal cu \$b și sunt de același tip. (PHP 4)
\$a != \$b	Diferit	TRUE dacă \$a nu este egal cu \$b.
\$a <> \$b	Diferit	TRUE dacă \$a nu este egal cu \$b.
\$a !== \$b	Non-identitate	TRUE dacă \$a nu este egal cu \$b sau nu sunt de același tip. (PHP 4)
\$a < \$b	Mai mic decât	TRUE dacă \$a este strict mai mic decât \$b.
\$a > \$b	Mai mare decât	TRUE dacă \$a este strict mai mare decât \$b.
\$a <= \$b	Mai mic sau egal cu	TRUE dacă \$a este mai mic sau egal cu \$b.
\$a >= \$b	Mai mare sau egal cu	TRUE dacă \$a este mai mare sau egal cu \$b.

Operatorul terțiar ?:

Un alt operator condițional este operatorul terțiar "?:". Expresia

```
(expr1) ? (expr2) : (expr3)
```

se evaluează la expr2 dacă expr1 este evaluată la valoarea TRUE și la expr3 dacă expr1 este evaluată la valoarea FALSE.

```
<?php
//Operator condițional terțiar "?:". Structura (expr1) ? (expr2) : (expr3)
$variabila = "CORECTA";
echo $variabila == "CORECTA" ? "Variabila are valoarea CORECTA" : "Variabila nu are valoarea CORECTA";
?>
```

Operatori de control al erorilor („@”)

Există un singur operator „@” care inhibă erorile ce pot apărea în cadrul expresiei pe care o prefixează.

Operatori de incrementare / decrementare ("++" și "--")

Exemple	Nume	Rezultat
++\$a	Pre-increment	Incrementează \$a cu 1, apoi întoarce \$a.
\$a++	Post-increment	Întoarce \$a, apoi incrementează \$a cu 1.
--\$a	Pre-decrement	Decrementează \$a cu 1, apoi întoarce \$a.
\$a--	Post-decrement	Întoarce \$a, apoi decrementează \$a cu 1.

Operatori logici (and sau "&&", or sau "||", xor, "!")

Exemple	Nume	Rezultat
\$a and \$b	And	TRUE dacă ambele \$a și \$b sunt TRUE.
\$a && \$b	And	TRUE dacă ambele \$a și \$b sunt TRUE.
\$a or \$b	Or	TRUE dacă fie \$a sau \$b sunt TRUE.
\$a \$b	Or	TRUE dacă fie \$a sau \$b sunt TRUE.
\$a xor \$b	Xor	TRUE dacă fie \$a, fie \$b sunt TRUE, dar nu simultan ambele.
! \$a	Not	TRUE dacă \$a nu este TRUE.

Operatori pentru șiruri de caractere

Operatorul de concatenare "." asemănător cu operatorul de concatenare din limbajul Perl.

```
<?php
//operatorul de concatenare "." pentru șiruri de caractere
$a = "Hello ";
$b = $a . "World!"; // $b conține "Hello World!"
$a = "Hello ";
$a .= "World!"; // $a conține de asemenea "Hello World!"
```

?>

Instrucțiunile limbajului PHP

Limbajul PHP pune la dispoziția programatorului instrucțiuni de atribuire (de forma \$variabilă = expresie), instrucțiuni de test (if și switch), instrucțiuni repetitive (while, do, for, foreach) și instrucțiuni de control (break, continue, return).

Structuri condiționale

Instrucțiunea if

```
<?php
if (expresie)
    instrucțiune
?>
```

Mai întâi, expresie (de tipul boolean) este evaluată la valoarea ei logică. Dacă expresie este evaluată la valoarea TRUE, atunci PHP va executa instrucțiune iar dacă expresie este evaluată la valoarea FALSE atunci PHP va ignora instrucțiune.

Instrucțiunea else

Clauza else este executată dacă expresia logică după if este evaluată la valoarea FALSE ca și toate expresiile logice elseif succesoare (dacă există). Altfel spus, toate expresiile logice predecesoare trebuie să fie evaluate la valoarea FALSE.

```
<?php
$a=5;$b=10;
if ($a > $b)
    { echo "a mai mare decat b";}
else
    { echo "a NU este mai mare decat b";}
?>
```

Instrucțiunea elseif

Instrucțiunea elseif este o combinație de if și else. Precum else, extinde o instrucțiune if pentru a executa o altă instrucțiune diferită în cazul în care expresia logică originală if este evaluată la FALSE. Totuși, spre deosebire de else, ramura alternativă va fi executată dacă și numai dacă expresia condițională elseif este evaluată la TRUE.

```
<?php
//Structura conditionala if..elseif..else
$a=5;$b=10;
if ($a > $b)
    {echo "a mai mare decat b";}
elseif ($a == $b)
    {echo "a este egal cu b";}
else
    {echo "a mai mic decat b";}
```

?>

Pot fi mai multe elseif în aceeași instrucțiune if. Prima alternativă elseif pentru care expresia logică este evaluată la TRUE va fi executată. În PHP, se poate scrie 'else if' (în 2 cuvinte) cu același comportament, identic ca și în cazul lui 'elseif' (într-un singur cuvânt).

Instrucțiunea switch

Instrucțiunea switch este asemănătoare cu o serie de instrucțiuni if cu aceeași expresie logică. În multe situații, se dorește compararea aceleiași variabile (sau expresii) cu mai multe valori diferite și să se execute secvențe diferite de cod depinzând de valoarea ei la un moment dat. Instrucțiunea structurată switch a fost proiectată tocmai pentru acest lucru.

Notă: Spre deosebire de alte limbaje de programare, instrucțiunea continue se aplică și acționează similar cu break. Dacă există un switch în interiorul unui ciclu și se dorește să se continue cu următoarea iterație a ciclului exterior, se va utiliza continue 2.

Următoarele 2 exemple ilustrează două modalități de a scrie același lucru: una prin utilizarea unei cascade de instrucțiuni if și elseif, și alta prin utilizarea instrucțiunii switch:

```
<?php
//Structura condicionala if..elseif si echivalentul ei switch..case
$i=0;
echo "STRUCTURA CONDIȚIONALĂ if..elseif<br />";
if ($i == 0) {
echo "i este egal cu 0";
} elseif ($i == 1) {
echo "i este egal cu 1";
} elseif ($i == 2) {
echo "i este egal cu 2";
}
echo "<br /><br />STRUCTURA CONDIȚIONALĂ switch..case<br />";
switch ($i) {
case 0:
    echo "i este egal cu 0";
    break;
case 1:
    echo "i este egal cu 1";
    break;
case 2:
    echo "i este egal cu 2";
    break;
}
?>
```

Pentru a evita greșelile este important de înțeles cum este executată instrucțiunea switch. Instrucțiunea switch se execută linie cu linie (de fapt, instrucțiune cu instrucțiune). La început, nu se execută nici un cod. Numai când se întâlnește o instrucțiune case cu o valoare care se potrivește valorii expresiei switch atunci PHP începe într-adevăr să execute instrucțiunile alternativei respective. PHP va continua să execute instrucțiunile până la sfârșitul blocului switch, sau până la prima apariție a instrucțiunii break. Dacă nu se adaugă o instrucțiune break la sfârșitul listei de instrucțiuni case, PHP va continua să execute instrucțiunile următoarei alternative case.

Exemplu - utilizarea structurii condiționale switch..case fără break

```
<?php
//Fara break la sfârșitul listei de instructiuni case...
//...PHP va continua sa execute instructiunile urmatoarei alternative case
$i=0;
switch ($i) {
case 0:
echo "i este egal cu 0<br />";
case 1:
echo "i este egal cu 1<br />";
case 2:
echo "i este egal cu 2<br />";
}
?>
```

În acest exemplu, dacă \$i este egal cu 0, PHP ar executa toate instrucțiunile echo. Dacă \$i este egal cu 1, PHP ar executa ultimele 2 instrucțiuni echo. Comportamentul dorit se obține numai dacă \$i este egal cu 2 (i.e. va fi afișat textul 'i este egal cu 2'). De aceea este important să nu se uite inserarea instrucțiunilor break în structura de ramificare multiplă switch. Într-o instrucțiune switch, condiția este evaluată numai o dată și rezultatul este comparat cu fiecare instrucțiune case. Într-o instrucțiune elseif, condiția este evaluată din nou. Dacă condiția este mai complicată decât o simplă comparație și/sau se află într-un ciclu foarte strâns, o instrucțiune switch poate fi mai rapidă. Lista de instrucțiuni pentru un case poate fi de asemenea vidă, ceea ce face să treacă controlul spre lista de instrucțiuni a următorului case.

Exemplu - utilizarea structurii condiționale switch..case cu alternative vide

```
<?php
//Structura case cu alternative vide...
$i=0;
switch ($i) {
case 0:
case 1:
case 2:
    echo "i este mai mic decat 3 dar nu negativ";
    break;
case 3:
    echo "i este 3";
}
?>
```

O situație specială o reprezintă alternativa case denumită default. Această alternativă corespunde oricăror altor cazuri care nu s-au potrivit celorlalte alternative case și trebuie să fie ultima instrucțiune (alternativă) case.

Exemplu - utilizarea structurii condiționale „switch..case” cu alternativa default

```
<?php
//Structura case cu alternativa default...
$i=3;
switch ($i) {
case 0:
    echo "i este egal cu 0";
    break;
case 1:
```

```

echo "i este egal cu 1";
    break;
case 2:
    echo "i este egal cu 2";
    break;
default:
    echo "i nu este egal cu 0, 1 sau 2";
}
?>

```

Expresia case poate fi orice expresie care poate fi evaluată la un tip simplu de dată: integer, floating-point numbers sau strings. Arrays sau objects nu pot fi utilizate decât dacă ele sunt redefinite ca tipuri simple. Se pot utiliza și alte sintaxe, alternative, pentru structurile de control ale ramurilor instrucțiunii structurate switch:

Exemplul - utilizarea structurii condiționale „switch..case” cu sintaxa alternativă ":" + endswitch

```

<?php
//Structura case cu sintaxe alternative ":" + endswitch;
$i=3;
switch ($i):
case 0:
    echo "i este egal cu 0";
    break;
case 1:
    echo "i este egal cu 1";
    break;
case 2:
    echo "i este egal cu 2";
    break;
default:
    echo "i nu este egal cu 0, 1 sau 2";
endswitch;
?>

```

Structuri repetitive

Instrucțiunea while

Ciclurile while reprezintă cele mai simple cicluri în PHP. Ele se comportă exact precum structurile similare din limbajul C. Structura de bază a sintaxei unei instrucțiuni while este:

```

while (expresie)
    instrucțiune;

```

Instrucțiunea while determină PHP să execute instrucțiunile incluse în mod repetat, atâta timp cât expresia while este evaluată la valoarea TRUE. Valoarea expresiei este verificată de fiecare dată la începutul ciclului, astfel încât chiar dacă această valoare se modifică în timpul execuției instrucțiunilor incluse, execuția nu se va opri până la sfârșitul iterației (rularea tuturor instrucțiunilor din cadrul ciclului). Atunci când expresia while este evaluată la valoarea FALSE de la început, instrucțiunile din cadrul ciclului nu vor fi executate nici măcar o dată.

Ca și în cazul instrucțiunii if, se pot grupa mai multe instrucțiuni în cadrul aceleiași instrucțiuni repetitive while prin încadrarea lor cu paranteze acoladă {...} sau prin utilizarea sintaxei alternative:

```
while (expr):
    instrucțiune ...
endwhile;
```

Exemplele următoare sunt identice și ambele afișează numerele de la 1 la 10:

Exemplul - utilizarea structurii repetitive while

```
<?php
/* exemplul 1 */
$i = 1;
while ($i <= 10) {
    echo $i++, " "; /* (post-increment) se afiseaza $i inainte de incrementare */
}
/* exemplul 2 */
echo "<br/>";
$i = 1;
while ($i <= 10):
    echo $i, " ";
    $i++;
endwhile;
?>
```

Instrucțiunea do..while

Ciclurile do..while sunt foarte asemănătoare ciclurilor while, cu excepția faptului că expresia logică este verificată la sfârșitul fiecărei iterații și nu la începutul ei. Principala diferență față de ciclurile while obișnuite este că prima iterație a ciclului do..while se execută întotdeauna (pentru că expresia logică este verificată de abia la sfârșitul ciclului). Acest comportament nu se întâlnește și la ciclurile while obișnuite (pentru că expresia logică este verificată la începutul fiecărei iterații iar dacă este evaluată la valoarea FALSE atunci execuția ciclului se încheie imediat).

Structura sintactică a unui ciclu do..while este:

```
<?php
$i = 0;
do {
    echo $i;
} while ($i > 0);
?>
```

Ciclul de mai sus este executat o singură dată deoarece după prima iterație, expresia logică este evaluată la valoarea FALSE (\$i nu este mai mare decât 0). Programatorii în C experimentați utilizează uneori în mod diferit ciclul do..while, permițând întreruperea execuției în mijlocul blocului de instrucțiuni, prin construirea unui ciclu infinit do..while și includerea instrucțiunii break precum în exemplul următor:

Exemplul - utilizarea structurii repetitive do..while ca un ciclu infinit

```
<?php
// ciclu infinit do..while
$factor=2;
$limita_minima=100;
$i=5; //se termina la a doua structura if
//$i=0; // se termina la prima structura if
do {
    if ($i < 5) {
        echo "Variabila i nu este suficient de mare";
        break;
    }
}
```

```

    }
    $i *= $factor;
    if ($i > $limita_minima) {
        echo "<br/>i=", $i, " i este prea mare!.. >", $limita_minima;
        break;
    }
    echo "<br/>i=", $i, " Variabila i este O.K.!", /* proceseaza i */
} while (1);
?>

```

Instrucțiunea for

Ciclurile for sunt cele mai complexe cicluri în PHP. Se comportă precum ciclul corespunzător din C. Sintaxa ciclului for este:

```

for (expr1; expr2; expr3) {
    instrucțiune
};

```

Prima expresie (expr1) este evaluată (executată) cel puțin o dată în mod necondiționat la începutul ciclului. La începutul fiecărei iterații, expr2 este evaluată. Dacă este evaluată la TRUE, ciclul continuă și instrucțiunile incluse sunt executate. Dacă în schimb este evaluată la FALSE, execuția ciclului se termină. La sfârșitul fiecărei iterații, expr3 este evaluată (executată). Fiecare din expresii poate fi vidă. Dacă expr2 este vidă aceasta înseamnă că ciclul ar trebui să se execute la infinit (PHP implicit consideră că este TRUE, precum în C). Aceasta poate fi uneori utilă atunci când se dorește terminarea ciclului prin utilizarea unei instrucțiuni condiționate break în locul expresiei logice a lui for. Toate exemplele următoare vor afișa numerele de la 1 la 10:

Exemplul - utilizarea structurii repetitive for pentru generarea și afișarea secvenței de numere 1..10.

```

<?php
// Generarea/afișarea secvenței de numere 1..10
echo "<br/> Exemplul 1 : ";
for ($i = 1; $i <= 10; $i++) {
    echo $i, " ";
}

echo "<br/> Exemplul 2 : ";
for ($i = 1; ; $i++) {
    if ($i > 10) { break; }
    echo $i, " ";
}

echo "<br/> Exemplul 3 : ";
$i = 1;
for (; ; ) {
    if ($i > 10) { break; }
    echo $i, " ";
    $i++;
}

echo "<br/> Exemplul 4 : ";
for ($i = 1; $i <= 10; $i++):
    echo $i, " ";
endfor;
?>

```

PHP permite de asemenea varianta alternativă a ciclului for prin utilizarea "sintaxei caracterului:"

```

for (expr1; expr2; expr3):

```

```
        instructiune; ...;
endfor;
```